# HDOC Day-55 Activity

**Maximum Element in Each Sliding Window Using Pointers and Nested Loops**

| | |
|---|---|
| **Student Name** | Dhruv Pandey |
| **SAP ID** | 590037660 |
| **Batch** | 15 |
| **Course / Activity** | HDOC Day-55 |

## Aim

To write a C program that considers every contiguous subarray of size **k** as a sliding window and displays the maximum element from each window. The input array is passed to **window_maximum()** using its address and accessed through pointers.

## Problem Requirements

- Read the size and elements of a variable-length integer array **arr[]**.
- Read an integer **k** such that 0 < k <= size.
- Pass the array to **window_maximum()** using its address.
- Consider every contiguous group of k elements as a sliding window.
- Find the maximum value in every window from left to right.
- Print the maximum values separated by spaces.
- Display **-1** when k is invalid.
- Use a nested-loop approach.

## Algorithm

1. Read the array size **n**, array elements, and **k**.
2. Pass **arr**, **n**, and **k** to **window_maximum()**.
3. If **k <= 0** or **k > n**, print -1 and return.
4. Use an outer loop from **i = 0** through **i = n-k**. Each i is the starting index of one sliding window.
5. Initialize the maximum value with the first element of the current window.
6. Use an inner loop to examine the remaining k-1 elements using pointer notation.
7. Update the maximum whenever a larger element is found.
8. Print the maximum for each window, separated by spaces.

## C Program

```c
#include <stdio.h>

void window_maximum(int *arr, int n, int k)
{
    if (k <= 0 || k > n)
    {
        printf("-1");
        return;
    }

    for (int i = 0; i <= n - k; i++)
    {
        int maximum = *(arr + i);

        for (int j = 1; j < k; j++)
        {
            if (*(arr + i + j) > maximum)
            {
                maximum = *(arr + i + j);
            }
        }

        printf("%d", maximum);

        if (i < n - k)
        {
            printf(" ");
        }
    }
}

int main()
{
    int n, k;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    window_maximum(arr, n, k);

    return 0;
}
```

## Pointer Usage

**window_maximum(arr, n, k)** passes the address of the first array element. The function parameter **int *arr** receives that address. The expression ***(arr + i + j)** accesses an element inside the current sliding window.

## Example 1 Dry Run

| Window | Elements | Maximum |
|---|---|---|
| 1 | 1 3 -1 | 3 |
| 2 | 3 -1 -3 | 3 |
| 3 | -1 -3 5 | 5 |
| 4 | -3 5 3 | 5 |
| 5 | 5 3 6 | 6 |
| 6 | 3 6 7 | 7 |

Therefore, for **1 3 -1 -3 5 3 6 7** with k = 3, the output is **3 3 5 5 6 7**.

## Test Cases / Expected Results

| Case | Input Array | k | Expected Output |
|---|---|---|---|
| Example 1 | 1 3 -1 -3 5 3 6 7 | 3 | 3 3 5 5 6 7 |
| Example 2 | 1 2 3 4 5 | 2 | 2 3 4 5 |
| Example 3 | 1 2 3 | 4 | -1 |

## Example 2 Explanation

For array **1 2 3 4 5** and k = 2, the windows are [1,2], [2,3], [3,4], and [4,5]. Their maximum values are 2, 3, 4, and 5. Hence the output is **2 3 4 5**.

## Example 3 - Invalid k

For an array of size 3, k = 4 is invalid because k is greater than the array size. Therefore, the program prints **-1**.

## Complexity

**Time Complexity:** O((n-k+1) x k), generally written as O(nk), because each of the n-k+1 windows examines up to k elements using nested loops.

**Space Complexity:** O(1) extra space because no additional array is required.

## Result

The program successfully processes every contiguous sliding window of size k, finds its maximum element using pointers and nested loops, prints the maximum values from left to right separated by spaces, and displays -1 for an invalid k.

## Execution Example

```
Enter the size of array: 8
Enter the elements: 1 3 -1 -3 5 3 6 7
Enter k: 3
3 3 5 5 6 7
```